Sylva — DeepForest Pipeline Setup Guide

Replaces: Workstream A (GroundingDINO + SAM2) **What it does:** Takes a GeoTIFF orthomosaic from your drone, detects every tree crown, and outputs per-tree analytics (location, canopy diameter, coloration, health indices).

Why DeepForest Instead of GroundingDINO + SAM2

DeepForest is purpose-built for aerial tree crown detection. The pretrained model was trained on over 30 million crowns from NEON (National Ecological Observatory Network) imagery at ~10 cm GSD — almost exactly what your drone setup will produce.

Compared to the previous stack:

	GroundingDINO + SAM2	**DeepForest**
Setup complexity	Clone repo, download checkpoints, build CUDA extensions, configure SAHI manually	`pip install deepforest`
Model	General-purpose zero-shot detector	Pretrained specifically on tree crowns
Tiling	Manual via `supervision.InferenceSlicer`	Built-in `predict_tile()` with NMS
Output	Bounding boxes + masks	Bounding boxes with confidence (masks via optional CropModel)
CPU speed	~5-15 sec/image per slice	~7 min per 1 km² tile on CPU

Prerequisites

Same as before — Python 3.10+ on Linux:

```
python3 --version   # should be 3.10, 3.11, or 3.12
pip --version
```

Step-by-Step Setup

Step 1: Use Your Existing Virtual Environment (or Create a New One)

If you already have the `~/sylva/venv` from Workstream A, you can reuse it:

```
cd ~/sylva
source venv/bin/activate
```

Or create a fresh one:

```
mkdir -p ~/sylva-deepforest
cd ~/sylva-deepforest
python3 -m venv venv
source venv/bin/activate
```

Step 2: Install PyTorch (CPU-Only)

Same CPU-only PyTorch as before:

```
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cpu
```

Step 3: Install DeepForest and Dependencies

```
pip install deepforest rasterio numpy opencv-python pandas matplotlib
```

That's it. No cloning repos, no checkpoint downloads, no compilation.

Step 4: Verify Installation

```
python3 -c "
from deepforest import main
model = main.deepforest()
print('DeepForest loaded successfully!')
print(f'Model config: {model.config}')
"
```

The first run downloads pretrained weights (~170 MB) to `~/.cache/`. Subsequent runs use the cache.

> **Note:** If you see examples online using `model.use_release()`, that method was deprecated in DeepForest 2.0. The pretrained tree model now loads automatically on init.

Step 5: Quick Test on a Sample Image

DeepForest ships with a built-in sample image:

```
python3 -c "
from deepforest import main
from deepforest import get_data

model = main.deepforest()
```

```
# Predict on built-in sample image
sample = get_data('OSBS_029.png')
boxes = model.predict_image(path=sample, return_plot=False)
print(boxes.head())
print(f'\nDetected {len(boxes)} trees')
"
```

Expected output: a DataFrame with columns `xmin`, `ymin`, `xmax`, `ymax`, `label`, `score`.

Running the Sylva Pipeline

Basic Usage

```
python sylva_deepforest_pipeline.py /path/to/your/orthomosaic.tif
```

With Custom Parameters

```
python sylva_deepforest_pipeline.py /path/to/orthomosaic.tif \
    --patch-size 400 \
    --patch-overlap 0.25 \
    --score-thresh 0.3 \
    --tile-size 2048 \
    --output-dir outputs/sylva_analysis
```

Quick Test (Process Only a Few Tiles)

```
python sylva_deepforest_pipeline.py /path/to/orthomosaic.tif --max-tiles 3
```

Pipeline Outputs

After running, you'll find these in the output directory:

```
outputs/sylva_analysis/
├── tree_inventory.csv          # Every detected tree with all metrics
├── tree_inventory.json         # Same data in JSON format
├── analysis_summary.json       # Aggregate statistics
└── analysis_plots.png          # 4-panel diagnostic visualization
```

Per-Tree Data (CSV/JSON columns)

Column	Description
`tree_id`	Unique integer ID
`score`	Detection confidence (0–1)
`px_xmin/ymin/xmax/ymax`	Bounding box in full-raster pixel coords
`crown_width_m`, `crown_height_m`	Crown dimensions in meters (if GSD available)
`crown_diameter_m`	Average of width and height in meters
`geo_x`, `geo_y`	Center of crown in the raster's CRS
`mean_r/g/b`	Average RGB channel values (0–255)
`greenness_exg`	Excess Green Index: 2G - R - B
`vari`	Visible Atmospherically Resistant Index: (G-R)/(G+R-B)
`brightness`	Mean luminance

Vegetation Indices Explained

ExG (Excess Green): Simple greenness measure. Higher = greener canopy. Useful for spotting dead/dying trees (low ExG) vs healthy ones (high ExG). Range is roughly -255 to +510 on a 0–255 scale.

VARI (Visible Atmospherically Resistant Index): More robust vegetation index using only RGB bands. Range is roughly -1 to +1. Values above 0 generally indicate green vegetation; values near 0 or negative may indicate bare soil, dead foliage, or non-vegetation.

These are computed from RGB only — for more precise health assessment, multispectral (NIR) data would be needed, but for a prototype demo with standard drone cameras, ExG and VARI are solid starting points.

Tuning Guide

DeepForest Parameters

Parameter	Default	Tuning Advice
`--patch-size`	400	Matches training resolution. Increase to 600–800 if your GSD is finer than 5 cm.
`--patch-overlap`	0.25	25% overlap. Increase to 0.35 if trees on patch boundaries are being missed.
`--score-thresh`	0.3	Lower (0.2) = more detections, more false positives. Higher (0.5) = fewer, more confident.

Tile Parameters (for Large Orthomosaics)

Parameter	Default	Tuning Advice

Parameter	Default	Tuning Advice
`--tile-size`	2048	How big a chunk to read from the GeoTIFF at a time. Increase if you have RAM.
`--tile-overlap`	128	Catches trees on tile boundaries. 128 px is safe for most crown sizes.

GSD Considerations

DeepForest was trained at ~10 cm/pixel. If your drone imagery has a very different GSD:

- **Finer than 5 cm/px:** Trees will appear very large relative to the 400px patch. Increase `--patch-size` to 600 or 800.
- **Coarser than 15 cm/px:** Trees may be too small to detect reliably. Consider flying lower or using a higher-resolution camera.
- **~8–12 cm/px:** Sweet spot — default parameters should work well.

Integration with Existing inspect_ortho.py

The pipeline replaces `inspect_ortho.py`'s tiling for detection purposes, but `inspect_ortho.py` is still useful for:

- **Quick metadata inspection:** `python inspect_ortho.py ortho.tif inspect`
- **Overview image generation:** `python inspect_ortho.py ortho.tif overview`
- **Sample region extraction:** `python inspect_ortho.py ortho.tif sample`

The DeepForest pipeline handles its own tiling internally.

What About Height?

DeepForest works on RGB only, so it can't directly measure tree height from a single orthomosaic. However, there are two paths forward:

1. **Stereo vision (your planned approach):** With two RPi Zero 2W cameras + OpenCV stereo, you can generate a DSM (Digital Surface Model) and extract height per tree using the bounding box locations from this pipeline.

2. **Photogrammetric DSM:** If your drone captures overlapping images, software like OpenDroneMap can generate a DSM alongside the orthomosaic. The pipeline's geo-located tree positions can then be cross-referenced against the DSM to extract height.

Both approaches use the `geo_x`, `geo_y` coordinates from this pipeline as the spatial key.

Next Steps

1. **Run on your actual orthomosaic** and evaluate detection quality
2. **Tune thresholds** based on your pine farm's density and GSD

3. **Pine tip moth detection:** DeepForest supports fine-tuning with custom annotations — you can annotate affected vs healthy trees and retrain
4. **Integrate with Workstream C:** The `geo_x`, `geo_y` output is your spatial anchor for GPS geolocation

---

File Structure After Setup

```
~/sylva-deepforest/
├── venv/                          # Python virtual environment
├── sylva_deepforest_pipeline.py   # Main pipeline script
├── inspect_ortho.py               # (optional) GeoTIFF inspection tool
└── outputs/
    └── sylva_analysis/
        ├── tree_inventory.csv
        ├── tree_inventory.json
        ├── analysis_summary.json
        └── analysis_plots.png
```